

现代编程思想

数据结构实现：队列和变长数组

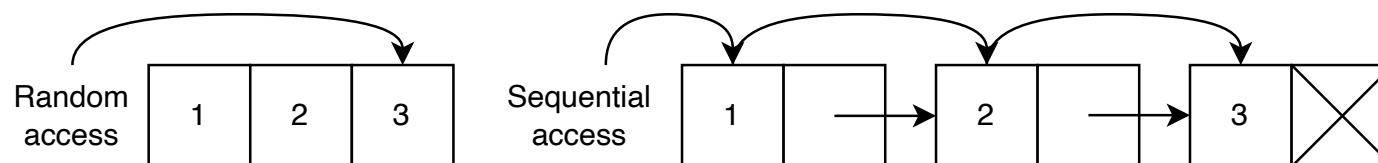
月兔公开课课程组

队列和变长数组

- 队列
 - 先进先出
 - 曾利用两个堆栈进行实现
 - 基于定长数组
- 变长数组
 - 随机存取
 - 可改变长度
 - 基于定长数组

定长数组

- 长度在创建时固定
- 可随机存取



```
1. fn[T] FixedArray::make(len : Int, init : T) -> Self[T]
2. fn[T] FixedArray::set(self : FixedArray[T], idx : Int, val : T) -> Unit
3. fn[T] FixedArray::get(self : FixedArray[T], idx : Int) -> T?
4. fn[T] FixedArray::length(self : FixedArray[T]) -> Int
```

- 例子:

```
1. test {
2.   let a : FixedArray[Int] = FixedArray::make(5, 0)
3.   a.set(0, 1)
4.   a.set(1, 2)
5.   inspect(a, content="[1, 2, 0, 0, 0]")
6. }
```

队列

- 队列接口定义:

```
1. fn[T] Queue::new() -> Queue[T]           // 创建空列表
2. fn[T] Queue::push(self : Queue[T], val: T) -> Unit // 队尾添加元素
3. fn[T] Queue::pop(self : Queue[T]) -> T?       // 头部取出元素
4. fn[T] Queue::peek(self : Queue[T]) -> T?      // 查看当前头元素
5. fn[T] Queue::length(self : Queue[T]) -> Int   // 查看队列长度
```

- 其中 `push` 和 `pop` 会修改 `self`，为了进行链式调用，我们可以使用级联语法:

```
1. test {
2.   inspect(Queue::new()..push(1)..push(2).length(), content="2")
3. }
```

- `x..f().g()` 等价于 `{ x.f(); x.g(); }`

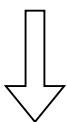
使用定长数组实现循环队列

new()

0

End

Start



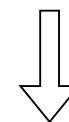
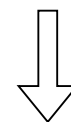
0	0	0	0	0
---	---	---	---	---

push(1)

1

Start

End



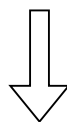
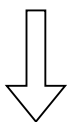
1	0	0	0	0
---	---	---	---	---

push(2)

2

Start

End



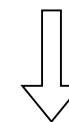
1	2	0	0	0
---	---	---	---	---

pop()

3

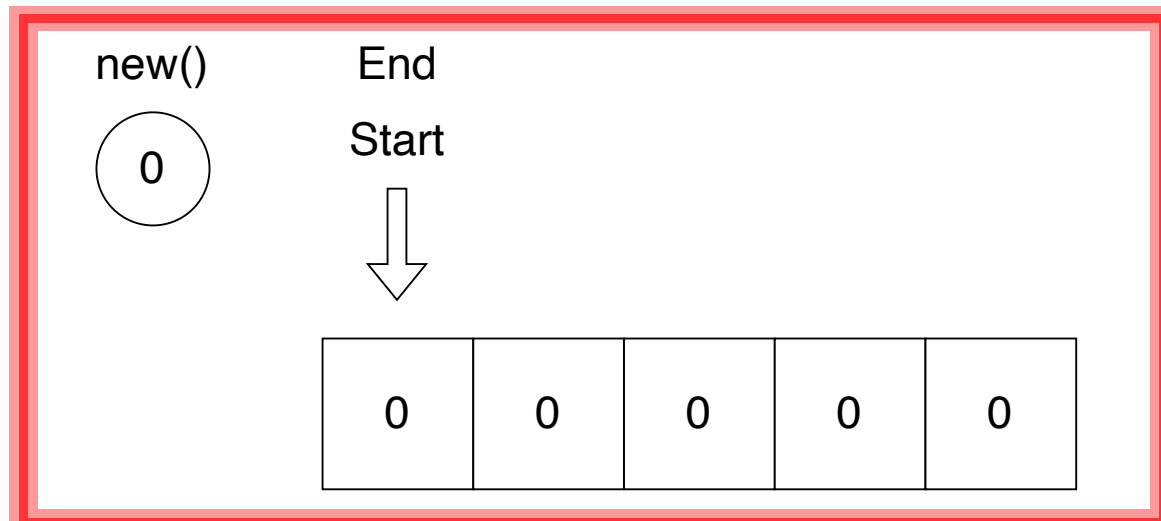
Start

End



0	2	0	0	0
---	---	---	---	---

使用定长数组实现循环队列



push(1)

1

Start

End

1	0	0	0	0
---	---	---	---	---

push(2)

2

Start

End

1	2	0	0	0
---	---	---	---	---

pop()

3

Start

End

0	2	0	0	0
---	---	---	---	---

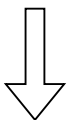
使用定长数组实现循环队列

new()

0

End

Start



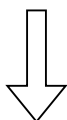
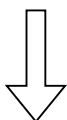
0	0	0	0	0
---	---	---	---	---

push(2)

2

Start

End



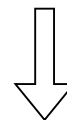
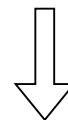
1	2	0	0	0
---	---	---	---	---

push(1)

1

Start

End



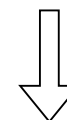
1	0	0	0	0
---	---	---	---	---

pop()

3

Start

End



0	2	0	0	0
---	---	---	---	---

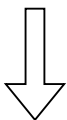
使用定长数组实现循环队列

new()

0

End

Start



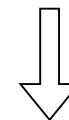
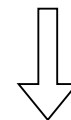
0	0	0	0	0
---	---	---	---	---

push(1)

1

Start

End



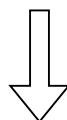
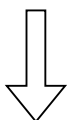
1	0	0	0	0
---	---	---	---	---

push(2)

2

Start

End



1	2	0	0	0
---	---	---	---	---

pop()

3

Start

End



0	2	0	0	0
---	---	---	---	---

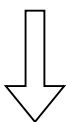
使用定长数组实现循环队列

new()

0

End

Start



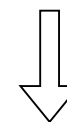
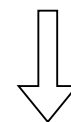
0	0	0	0	0
---	---	---	---	---

push(1)

1

Start

End



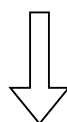
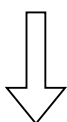
1	0	0	0	0
---	---	---	---	---

push(2)

2

Start

End



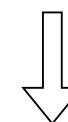
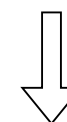
1	2	0	0	0
---	---	---	---	---

pop()

3

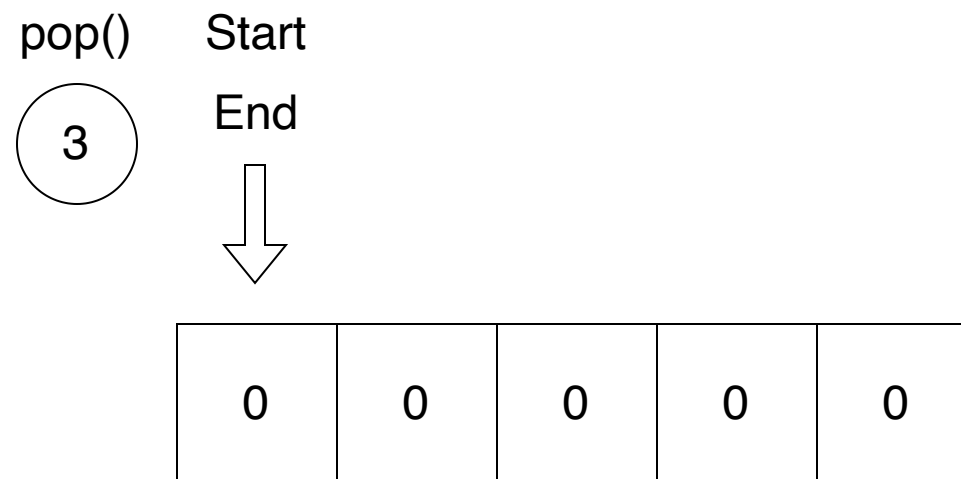
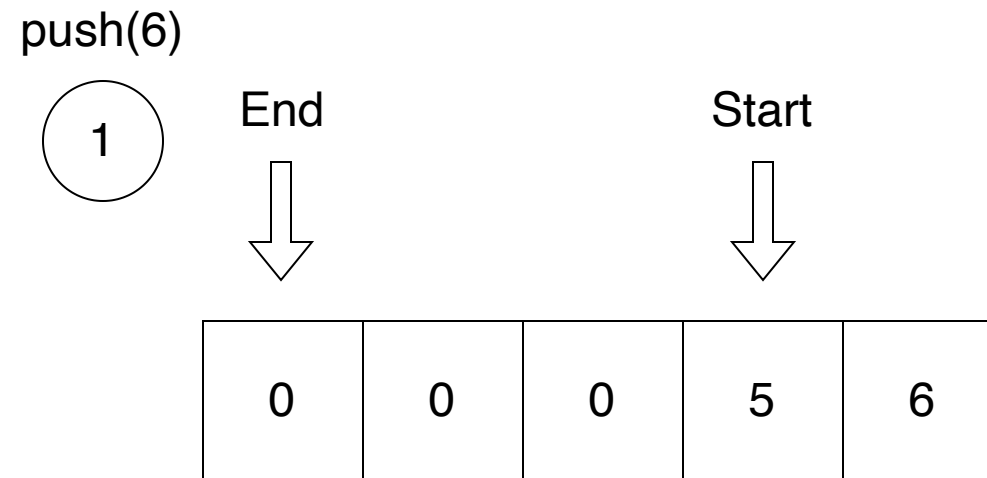
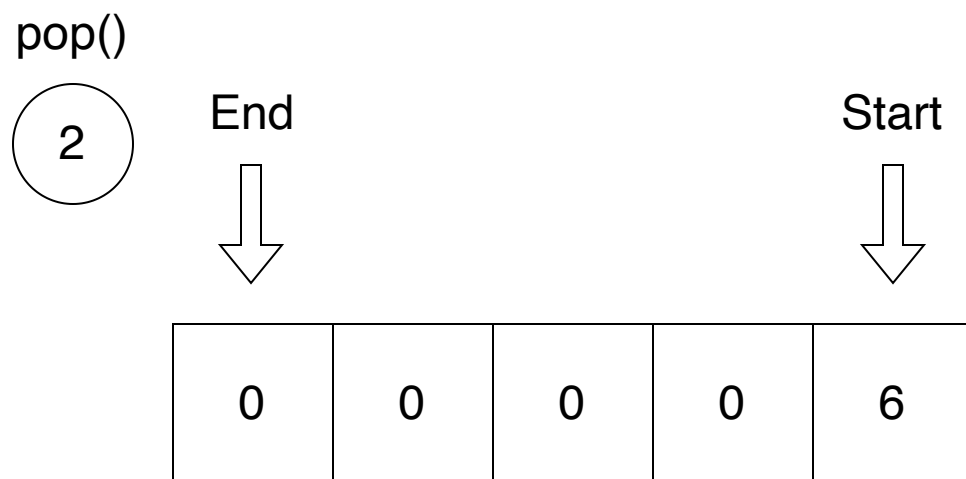
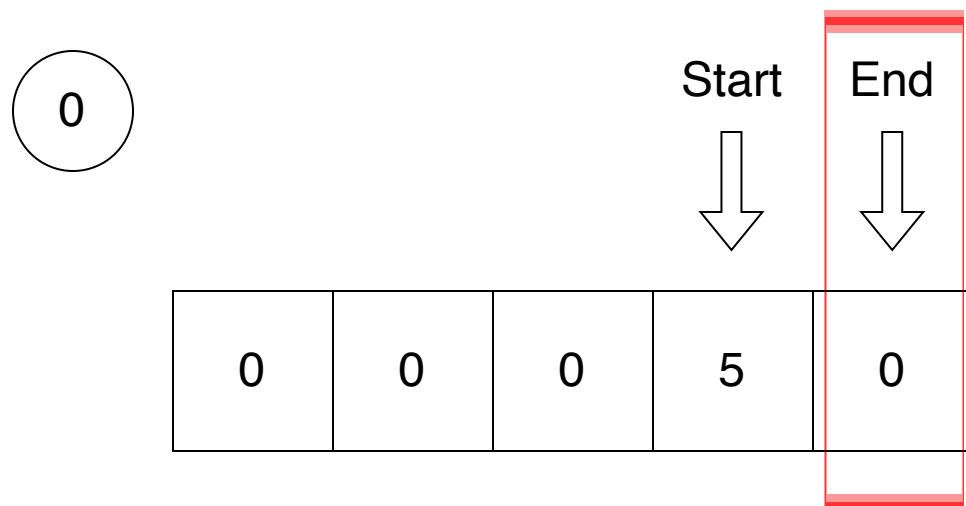
Start

End

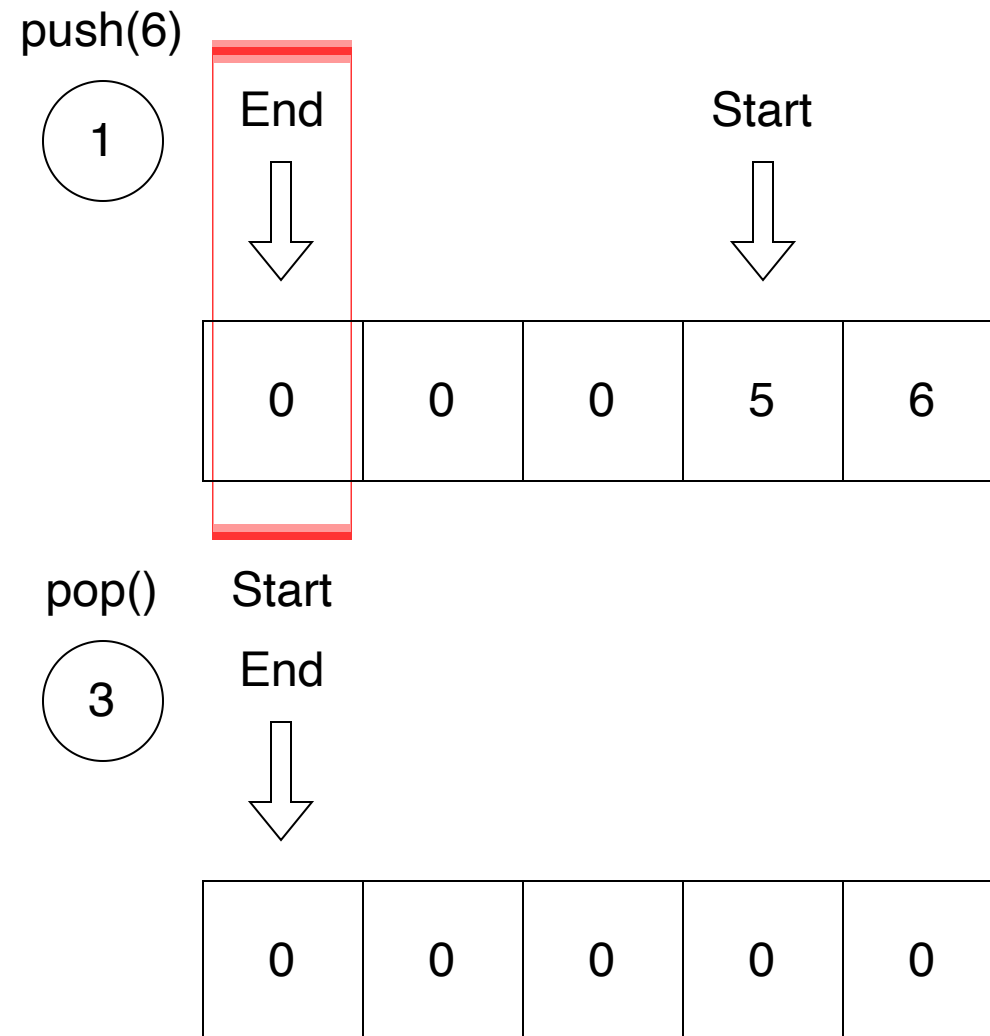
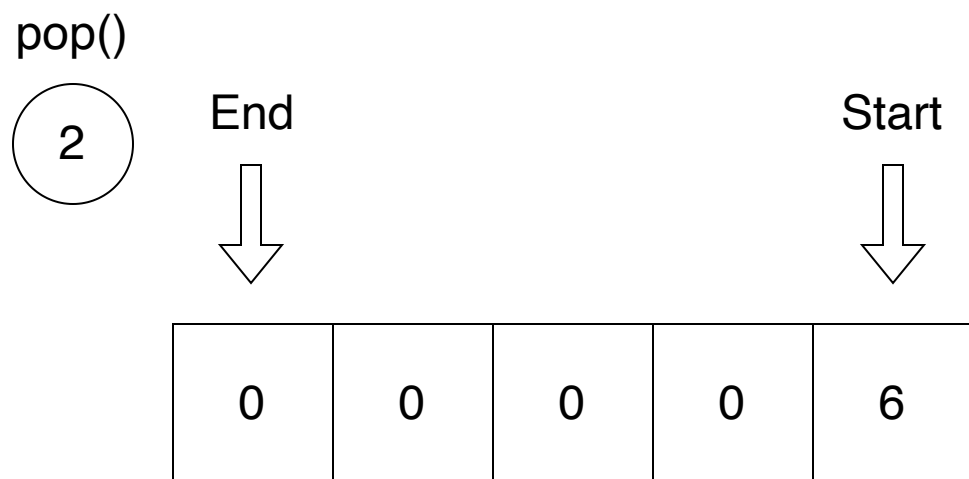
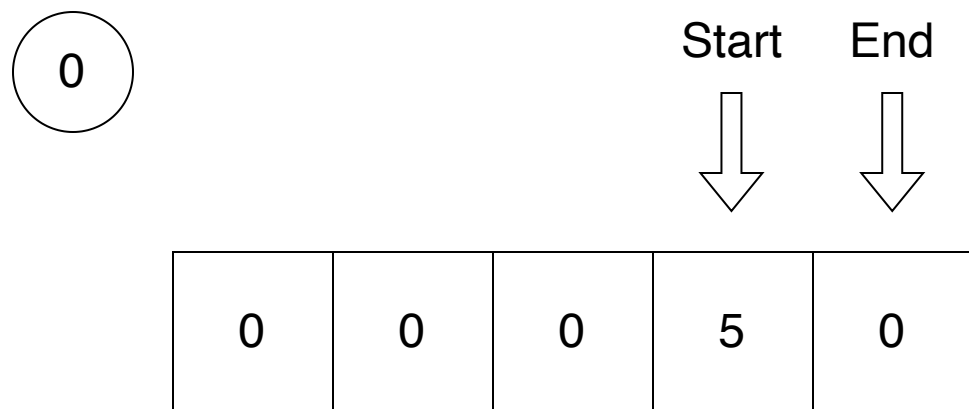


0	2	0	0	0
---	---	---	---	---

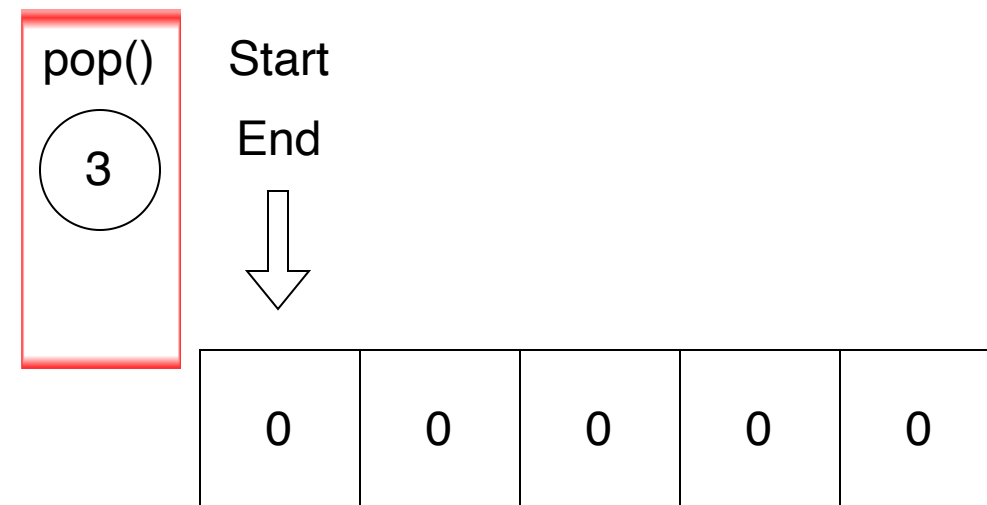
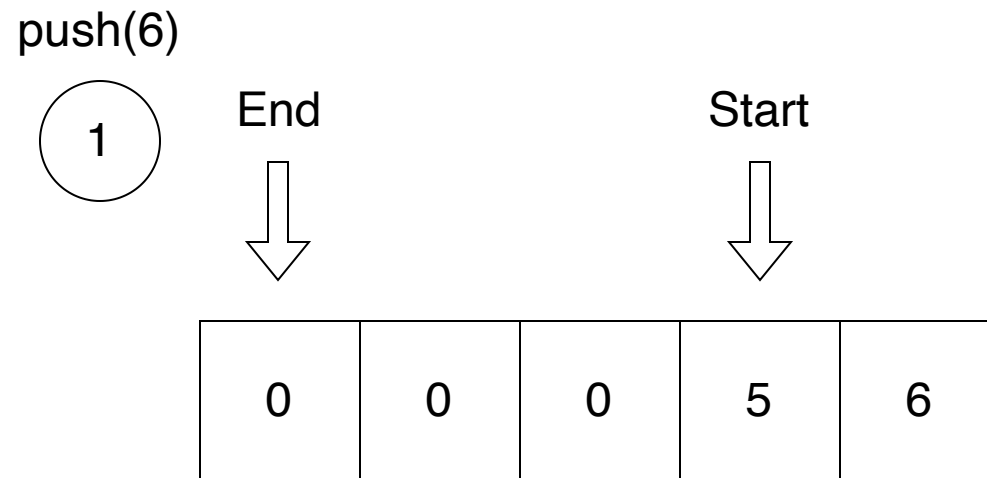
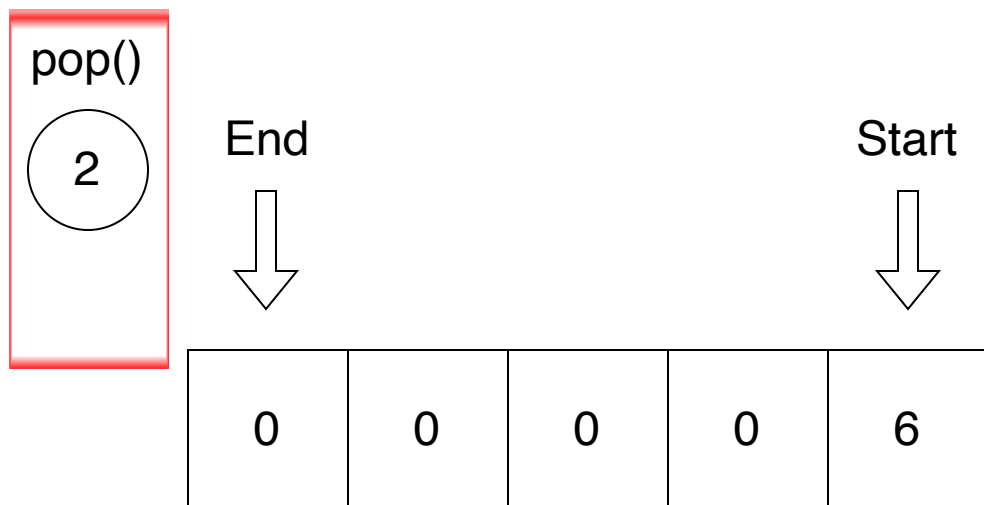
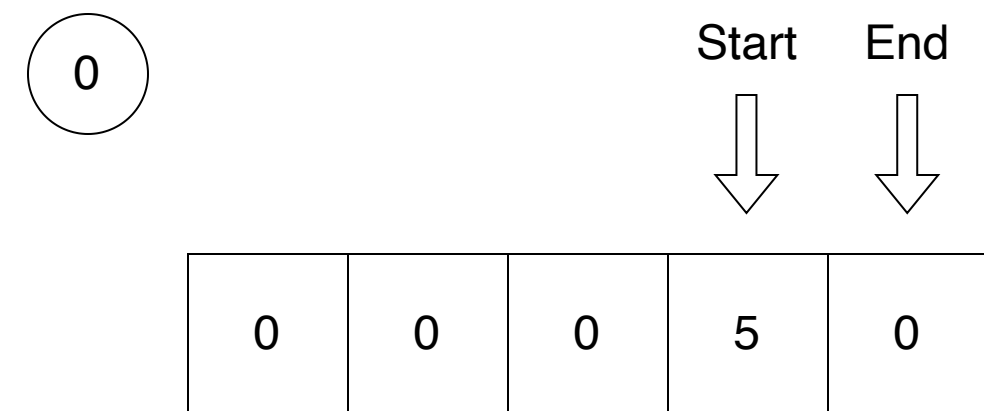
使用定长数组实现循环队列



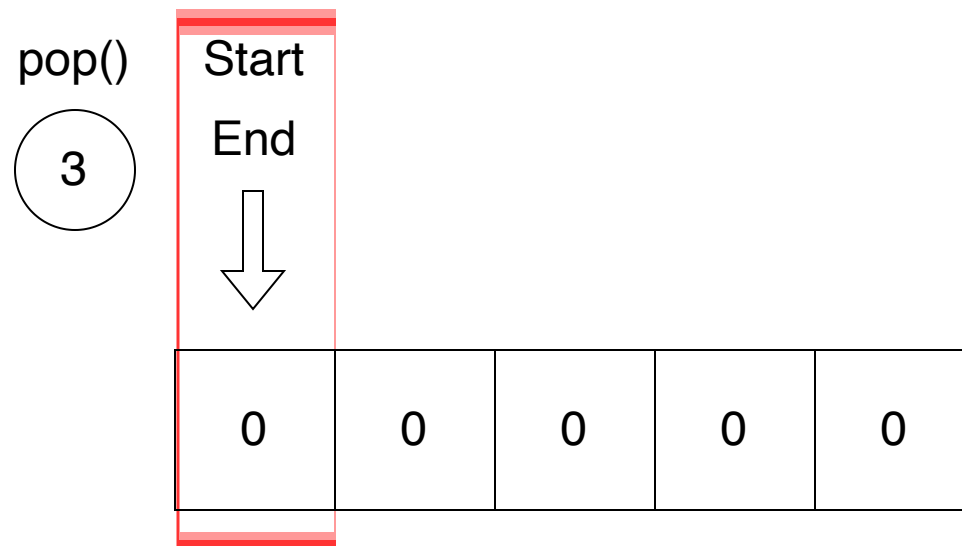
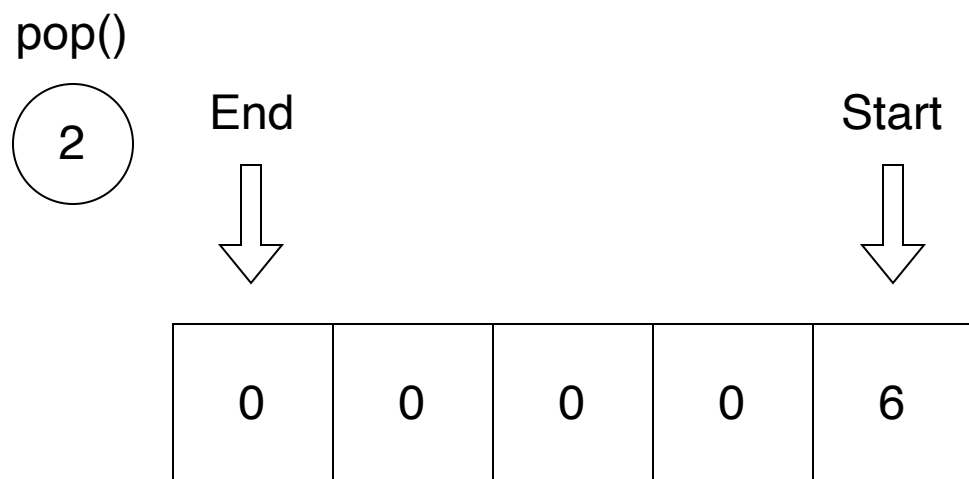
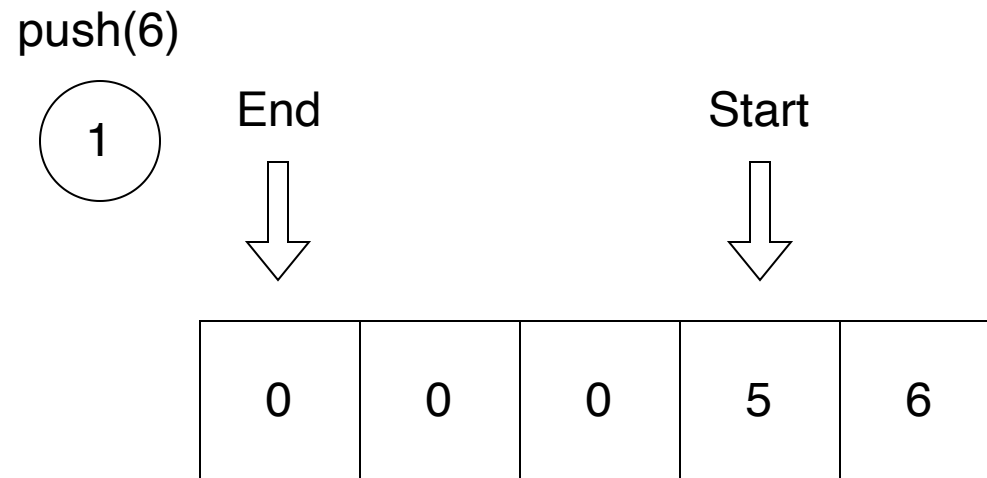
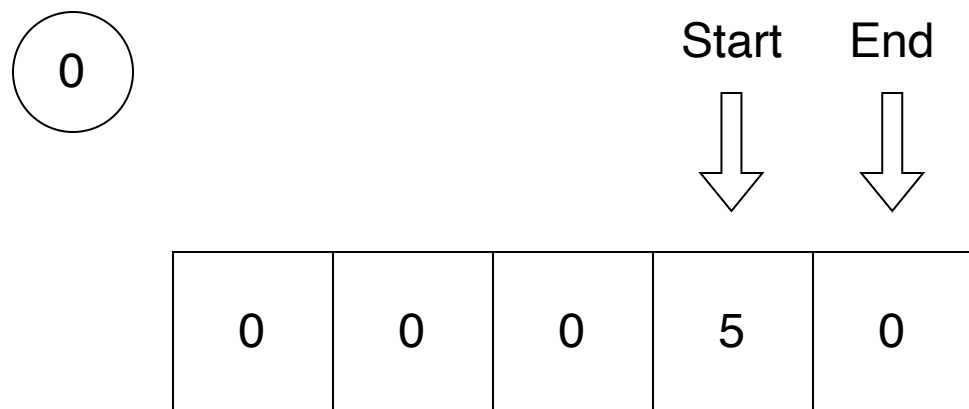
使用定长数组实现循环队列



使用定长数组实现循环队列



使用定长数组实现循环队列



循环队列

- 结构体定义

```
1. struct Queue[T] {  
2.     mut array : FixedArray[T]  
3.     mut start : Int  
4.     mut end : Int // end 指向队尾的后一个空块  
5.     mut length : Int  
6. }  
7.  
8. fn[T] Queue::length(self : Queue[T]) -> Int {  
9.     self.length  
10. }
```

循环队列：泛型和默认值

- 使用 `Default` 特征为类型提供默认值

```
1. fn [T : Default] Queue::new() -> Queue[T] {  
2.     {  
3.         array: FixedArray::make(8, T::default()),  
4.         start: 0,  
5.         end: 0,  
6.         length: 0,  
7.     }  
8. }
```

- `Int`、`String`、`Bool` 等都实现了 `Default` 特征
- 队列的长度此时为 0，而非 8

循环队列：添加元素

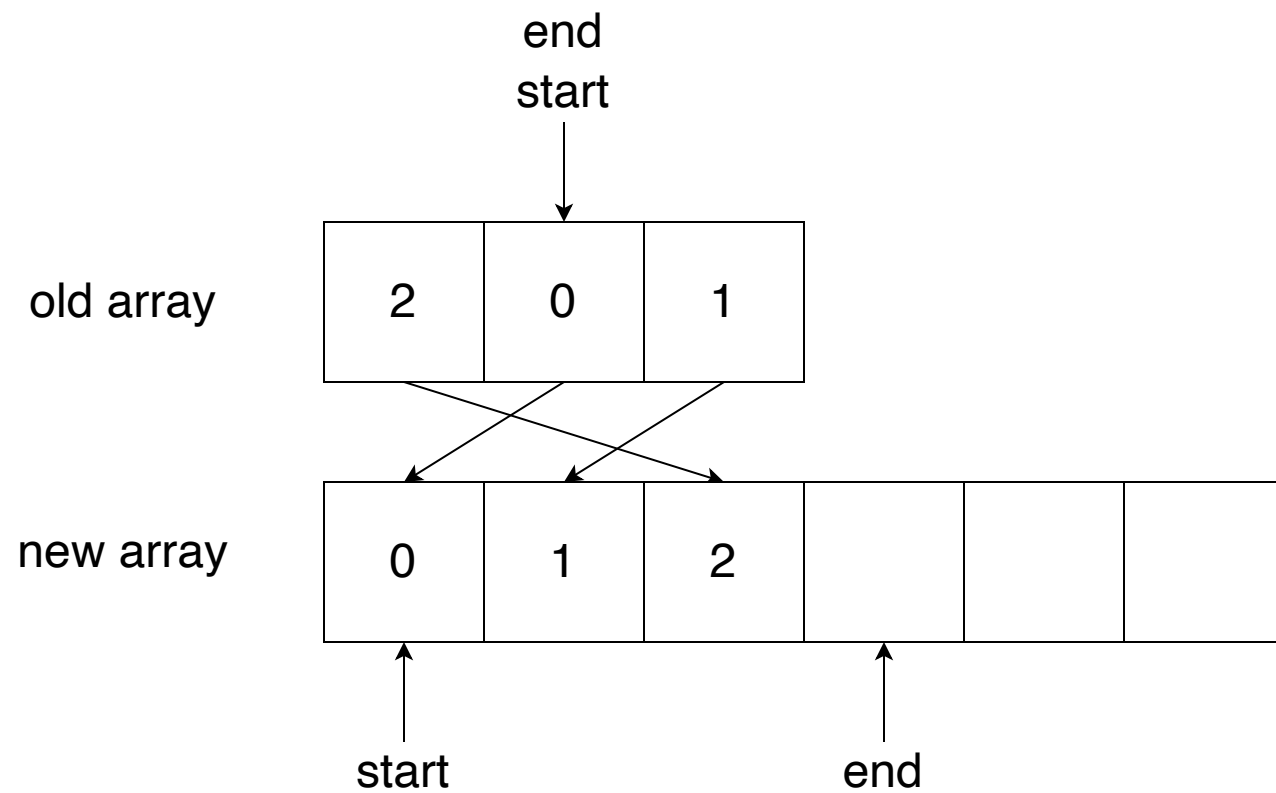
```
1. fn[T] Queue::naive_push(self : Queue[T], val : T) -> Unit {  
2.   self.array.set(self.end, val)  
3.   self.end = (self.end + 1) % self.array.length() // 超出队尾则转回队首  
4.   self.length = self.length + 1  
5. }
```

- 问题：如果元素数量超出了数组长度

循环队列：扩容

```
1. fn[T : Default] Queue::push(self : Queue[T], val : T) -> Unit {
2.   if self.length == self.array.length() { // 判断是否需要扩容
3.     let new_array = FixedArray::make(self.length * 2, T::default())
4.     for i = 0; i < self.array.length(); i = i + 1 {
5.       new_array[i] =
6.         self.array[(self.start + i) % self.array.length()]
7.     } // 将原数组的元素逐个复制到新的数组
8.     self.start = 0
9.     self.end = self.array.length()
10.    self.array = new_array
11.  }
12.  self.naive_push(val)
13. }
```

循环队列：扩容



循环队列：取出元素

- `peek` 操作返回 `start` 指向的元素
- 队列为空时，返回 `None`
- `pop` 操作在 `peek` 操作的基础上将 `start` 后移

```
1. fn[T] Queue::peek(self : Queue[T]) -> T? {
2.     if self.length() == 0 { None }
3.     else { self.array.get(self.start) }
4. }
5.
6. fn[T : Default] Queue::pop(self : Queue[T]) -> T? {
7.     let val = self.peek()
8.     if val is Some(_) {
9.         self.array[self.start] = T::default()
10.        self.start = (self.start + 1) % self.array.length()
11.        self.length = self.length - 1
12.    }
13.    val
14. }
```

变长数组

- 变长数组在定长数组的基础上增添 `push` 和 `insert` 操作

```
1. fn[T] MyArray::make(len : Int, init : T) -> MyArray[T]
2. fn[T] MyArray::length(self : MyArray[T]) -> Int
3. fn[T] MyArray::set(self : MyArray[T], idx : Int, val : T) -> Unit
4. fn[T] MyArray::get(self : MyArray[T], idx : Int) -> T?
5. fn[T] MyArray::push(self : MyArray[T], val : T) -> Unit
6. fn[T] MyArray::insert(self : MyArray[T], idx : Int, val : T) -> Unit
7. fn[T] MyArray::each(self : MyArray[T], f : (T) -> Unit) -> Unit
```

变长数组：初始化

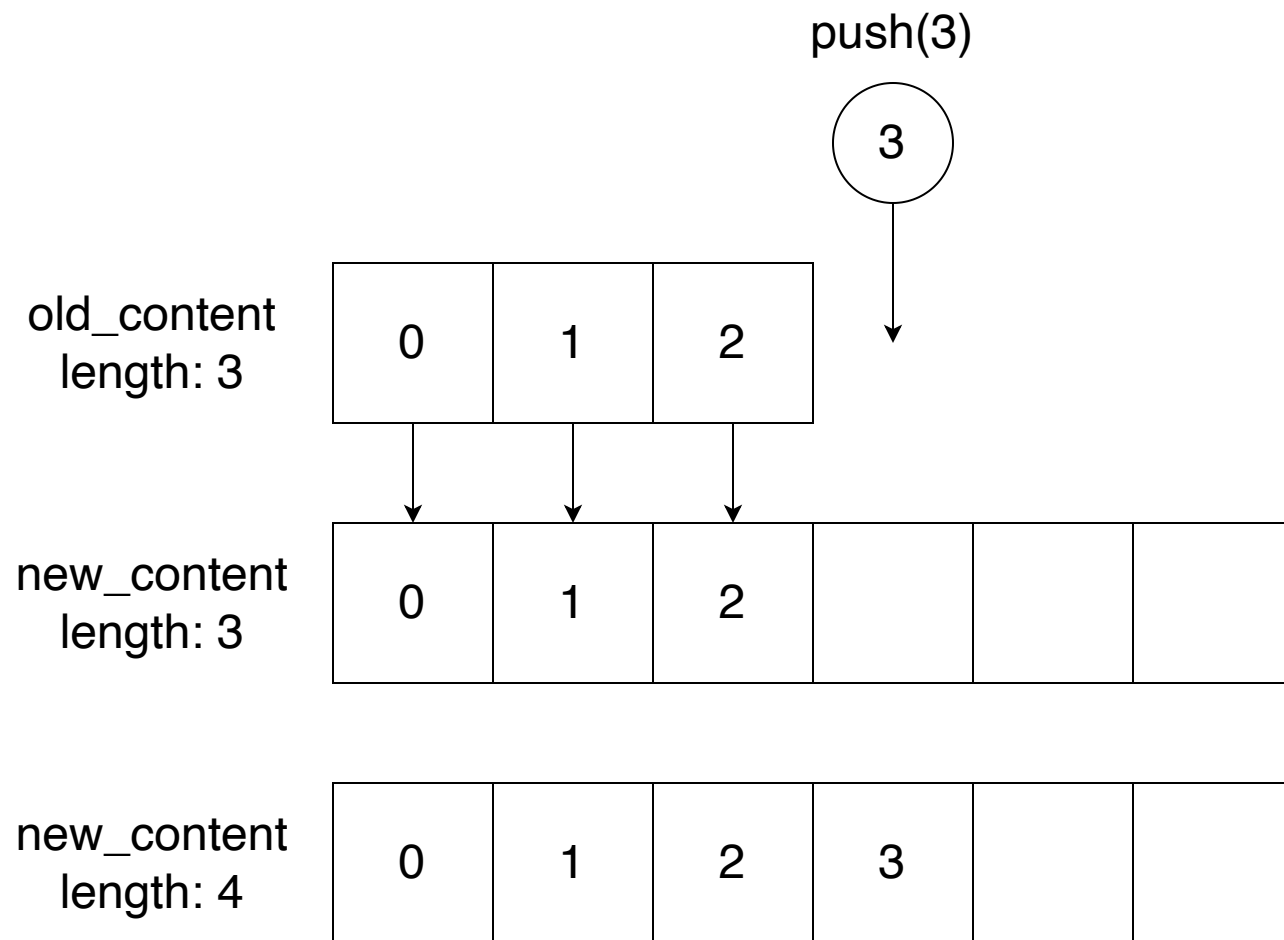
```
1. struct MyArray[T] {  
2.     mut content : FixedArray[T]  
3.     mut length : Int  
4. }  
5.  
6. fn[T] MyArray::make(len : Int, init : T) -> MyArray[T] {  
7.     {  
8.         content: FixedArray::make(len, init),  
9.         length: len  
10.    }  
11. }  
12.  
13. fn[T] MyArray::length(self : MyArray[T]) -> Int {  
14.     self.length  
15. }
```

变长数组：随机存取

- 在 `[0, length)` 区间外的写入是非法的

```
1. fn[T] MyArray::set(self : MyArray[T], idx : Int, val : T) -> Unit {
2.   guard idx >= 0 && idx < self.length
3.   self.content.set(idx, val)
4. }
5.
6. fn[T] MyArray::get(self : MyArray[T], idx : Int) -> T? {
7.   guard idx >= 0 && idx < self.length else { None }
8.   self.content.get(idx)
9. }
```

变长数组：扩容



变长数组：扩容

- 扩容可以单独抽取成一个函数
- 扩容后也不可访问 `[0, length)` 之外的元素

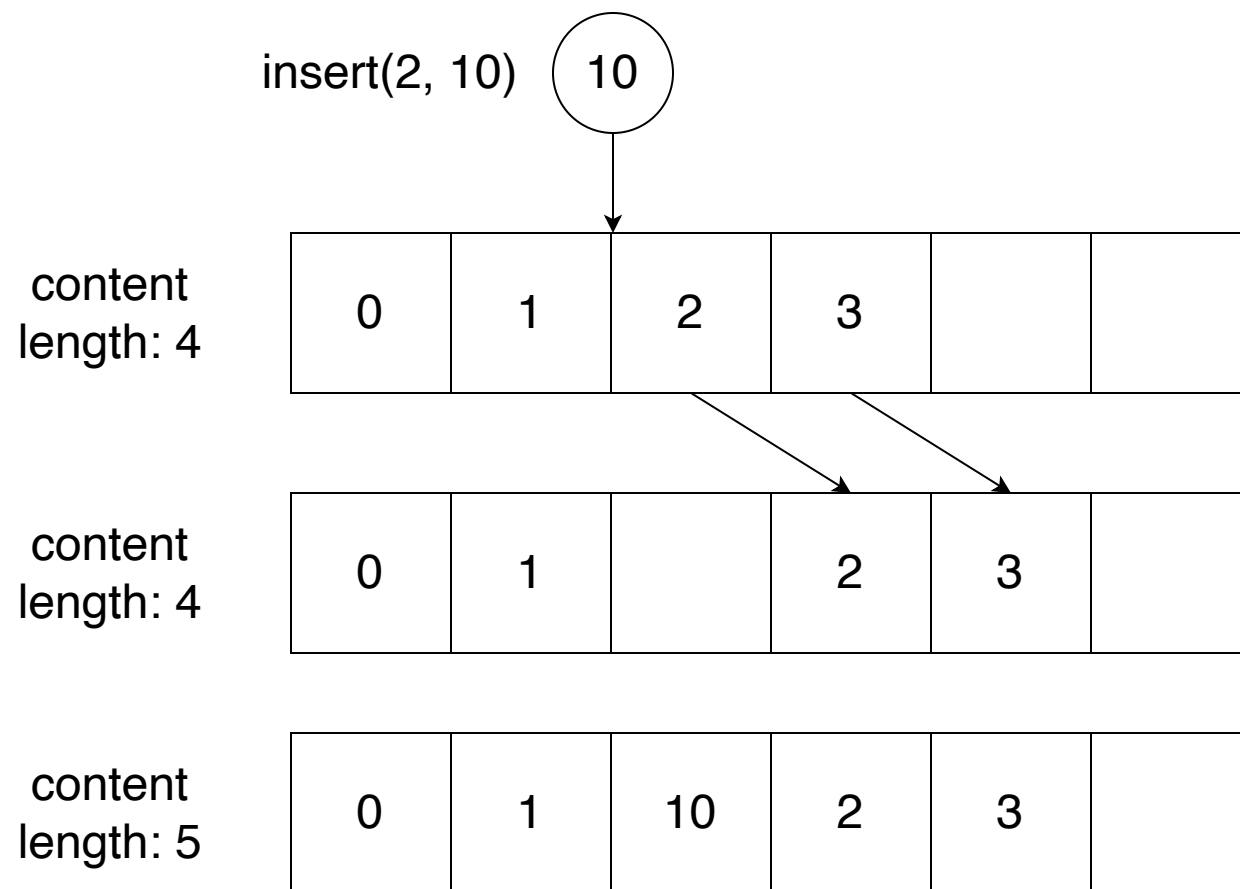
```
1. fn[T : Default] MyArray::realloc(self : MyArray[T]) -> Unit {
2.     let old_cap = self.length()
3.     let new_cap = if old_cap == 0 { 8 } else { old_cap * 2 }
4.     let new_content = FixedArray::make(new_cap, T::default())
5.     for i = 0; i < old_cap; i = i + 1 {
6.         new_content.set(i, self.content[i])
7.     }
8.     self.content = new_content
9. }
```


变长数组：尾部添加元素

- `push` 操作和循环队列类似：

```
1. fn[T : Default] MyArray::push(self : MyArray[T], val : T) -> Unit {  
2.     if self.length == self.content.length() {  
3.         self.realloc()  
4.     }  
5.     self.content.set(self.length, val)  
6.     self.length = self.length + 1  
7. }
```

变长数组：插入元素



变长数组：插入元素

- 将原定长数组的部分元素集体后移一位

```
1. fn[T : Default] MyArray::insert(self : MyArray[T], idx : Int, val : T) -> Unit {  
2.   if self.length == self.content.length() {  
3.     self.realloc()  
4.   }  
5.   for i = self.length - 1; i >= idx; i = i - 1 {  
6.     self.content.set(i + 1, self.content[i])  
7.   }  
8.   self.content.set(idx, val)  
9.   self.length = self.length + 1  
10. }
```

变长数组：遍历访问

```
1. fn[T] MyArray::each(self : MyArray[T], f : (T) -> Unit) -> Unit {
2.   for i = 0; i < self.length; i = i + 1 {
3.     let elem = self.get(i)
4.     guard elem is Some(elem)
5.     f(elem)
6.   }
7. }
8.
9. test {
10.   let arr1 = MyArray::make(5, 0)
11.   let arr2 = MyArray::make(5, 0)
12.   arr1..push(1)..push(2)..push(3)
13.   arr1.each(fn(elem) { arr2.insert(0, elem)} )
14.   inspect(arr2.get(0), content="Some(3)")
15.   inspect(arr2.get(1), content="Some(2)")
16.   inspect(arr2.get(2), content="Some(1)")
17. }
```

变长数组：课后练习

- 标准库的 `Array` 类型提供了非常丰富的操作，包括：
 - `remove` 移除某一个位置的元素
 - `append/add` 合并两个数组
 - `map` 将原来数组的元素一一映射到新的数组
- 以上功能可以自行实现

总结

- 本章节我们介绍了使用可变数据结构
 - 定长数组
 - 基于定长数组实现循环队列
 - 基于定长数组实现变长数组